

## Trabalho - Algoritmos e Lógica de Programação

Aluno: Roney de Lima Melo

RA: 3000000280641

Disciplina/Atividade: Algoritmos e Lógica de Programação

### Contextualização do desafio

Em uma linha de montagem, cada peça que sai fora dos parâmetros de peso, cor ou dimensão e não é barrada a tempo se transforma em retrabalho, devolução, parada de linha ou, no pior caso, um produto defeituoso que chega ao cliente final. Quando essa inspeção é feita manualmente como descrito no enunciado do desafio o processo fica sujeito a três problemas recorrentes:

- **Atrasos:** um inspetor humano tem limite de peças que consegue avaliar por hora, o que cria gargalos quando a produção acelera.
- **Falhas de conferência:** critérios aplicados "de cabeça" variam de pessoa para pessoa e de turno para turno, gerando inconsistência entre peças que deveriam ser avaliadas pela mesma régua.
- **Aumento no custo de operação:** retrabalho, devoluções e mão de obra dedicada à conferência elevam o custo por unidade produzida.

A automação ataca justamente esses três pontos: um critério de aprovação codificado é aplicado sempre da mesma forma, a avaliação de uma peça leva milissegundos, e o resultado (aprovada/reprovada, com motivo) fica disponível imediatamente para quem decide o que fazer com a peça. Este protótipo simula essa etapa de inspeção digital sem eliminar a peça física da linha, mas eliminando a subjetividade e a lentidão da decisão sobre ela.

### Estruturação do raciocínio lógico

O sistema foi dividido em módulos, cada um responsável por uma parte do fluxo (ver `../src/main.py`) e o restante de `../src`). Essa separação por responsabilidade foi a primeira decisão de design: em vez de um único script sequencial, cada regra de negócio vive em seu próprio arquivo, o que facilita testar e alterar uma parte sem afetar as demais.

**Repetição** — O programa é orientado a um menu que roda em laço ("**while True**" em `../src/main.py`), lendo a opção do usuário a cada iteração até que a opção "**0**" seja escolhida. Esse é o mesmo padrão usado dentro de cada funcionalidade sempre que é preciso validar uma entrada (por exemplo, insistir até o usuário digitar um número válido de peso ou comprimento).

**Decisão (condições)** — A regra de aprovação, em (`../src/inspecao.py`), é o núcleo do raciocínio condicional do projeto: a peça só é aprovada se **todos** os três critérios forem verdadeiros ao mesmo tempo (**peso entre 95g–105g, cor azul ou verde, comprimento entre 10cm–20cm**). Cada critério é checado de forma independente, e cada falha é acumulada em uma lista de motivos, assim uma peça pode ser reprovada por mais de um motivo simultaneamente, e o usuário vê exatamente qual regra ela violou, não apenas que ela "falhou".

**Funções** — Cada operação do menu foi isolada em uma função com uma única responsabilidade: `"avaliar_peca"` decide aprovação/reprovação, `"cadastrar_peca"` orquestra a entrada de dados e chama a avaliação, `"GerenciadorCaixas"` encapsula a lógica de abrir/fechar caixas, `"listar_pecas"`, `"listar_caixas_fechadas"`, `"remover_peca"` e `"gerar_relatorio"` cuidam cada uma de uma consulta ou ação sobre os dados já armazenados. Isso evita duplicar lógica de validação em vários lugares do código e torna o menu principal em `"main.py"` curto e legível ele apenas direciona a opção escolhida para a função correspondente.

**Estruturas de dados** — Peças e caixas são modeladas como `"dataclass"` em (`../src/models.py`). Uma caixa mantém sua própria lista de peças e um estado "fechada"; ao adicionar uma peça, ela verifica sua própria capacidade e se fecha automaticamente quando atinge o limite de 10 peças a regra de "quando fechar a caixa" fica junto do dado que ela descreve, em vez de espalhada pelo menu.

## Benefícios percebidos e desafios enfrentados

### Benefícios:

- Critério de aprovação centralizado em um único ponto (`"inspecao.py"`), o que evita divergência de regras entre as funcionalidades.
- Separação de módulos por responsabilidade tornou fácil adicionar a funcionalidade de remoção depois que o cadastro e a listagem já existiam, sem precisar alterar a lógica de avaliação.
- Feedback imediato e específico ao usuário: em vez de **"peça reprovada"**, o sistema informa exatamente quais critérios falharam e com quais valores, o que facilitaria num cenário real o ajuste do processo produtivo.

### Desafios enfrentados:

- **Definir a regra de remoção de peças aprovadas:** como a caixa fechada representa um lote já consolidado (análogo a um lote fisicamente lacrado/expedido em uma linha real), foi preciso decidir que peças aprovadas só podem ser removidas enquanto a caixa ainda está aberta depois de fechada, alterar seu conteúdo deixaria de refletir a realidade que a caixa representa.

- **Tratar a entrada de dados do usuário no terminal:** (peso e comprimento como número, aceitando tanto ponto quanto vírgula como separador decimal) sem travar o programa quando o valor digitado é inválido resolvido em (`../src/utlis.py`) com validação e repetição da pergunta até receber um valor aceitável. Para validação utilizei boa prática para tratamento de erros com `try except`.

- **Garantir que peças com múltiplos problemas:** (por exemplo, peso e cor fora do padrão ao mesmo tempo) reporta todos os motivos, e não apenas o primeiro encontrado, o que levou à decisão de acumular motivos em uma lista em vez de retornar no primeiro critério que falha.

## **Reflexão final: expansão para um cenário real**

Este protótipo representa a decisão lógica de aprovação e o controle de lote de forma isolada, com entrada manual via terminal. Para evoluir para um cenário industrial real, os pontos naturais de expansão seriam:

- **Sensores:** substituir a entrada manual (`"input()"`) por leitura direta de sensores de peso, visão computacional para cor e sensores de distância/laser para comprimento, alimentando `"avaliar_peca"` com dados capturados em tempo real na esteira, sem intervenção humana.

- **Integração industrial:** conectar o sistema a um CLP (Controlador Lógico Programável) ou a um barramento industrial (OPC-UA, Modbus) para que a decisão de aprovação acione fisicamente um atuador por exemplo, desviar a peça reprovada para uma esteira separada e para que o fechamento de uma caixa dispare um evento para o sistema de logística (WMS/ERP) da fábrica.

- **Persistência e rastreabilidade:** trocar as listas em memória por um banco de dados, permitindo consultar o histórico de peças e caixas por turno, lote ou período, e cruzar dados de qualidade ao longo do tempo.

- **IA/Machine Learning:** um modelo treinado sobre o histórico de peças aprovadas/reprovadas poderia ir além do critério fixo por faixa e detectar padrões de desvio antes que uma peça saia da faixa aceitável (manutenção preditiva), ou usar visão computacional com aprendizado profundo para inspecionar defeitos que não se resumem a peso/cor/comprimento (trincas, deformações, acabamento).

- **Dashboards e alertas em tempo real:** expor os relatórios (hoje impressos no terminal) como um painel web ou dashboard atualizado ao vivo, com alertas automáticos quando a taxa de reprovação de um motivo específico ultrapassar um limite, permitindo à equipe de qualidade agir antes que o problema se acumule.

Em resumo, a lógica implementada aqui avaliação por critérios, agrupamento em lotes de capacidade fixa e geração de relatórios consolidados é o núcleo de decisão que continuaria existindo em uma solução real; o que mudaria é a origem dos dados (sensores em vez de teclado) e o destino das decisões (atuadores e sistemas corporativos em vez de apenas texto no terminal).

## **Referências**

[Phyton Documentação](#)

[Python](#)

[Github](#)